ARKANOID in Python

How I built and extended a classic arcade game with pygame

▶ PRESS SPACE TO START

Navigate with space or →

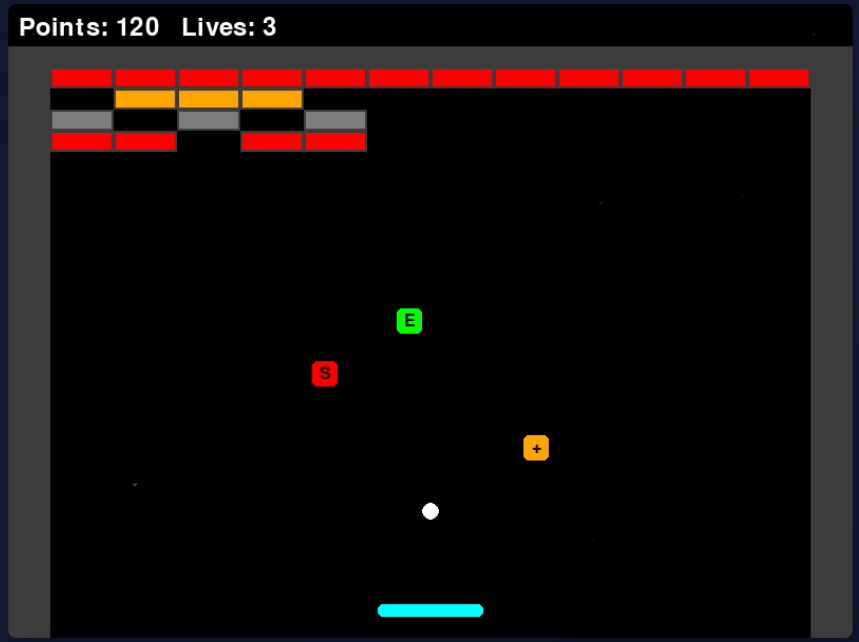
Behruz · 2026

Contents

Impatient? Jump straight to the demo

About the project

- **Python 3.11 + pygame 2.6.1**
- ~870 lines of code
- Levels defined in plain text files
- 7 kinds of power-ups
- Sound, particles, ball trail
- Code on GitHub: Sarvarchik2/python-game



Anatomy of the game

- **Paddle** — the player, moves with ← → arrow keys
- **Ball** — flies on its own, bounces off everything
- **Bricks** — take 1–2 hits, some are indestructible
- **Power-ups** — drop from destroyed bricks with a 30% chance
- **Laser** — shoots if you catch the L power-up
- **HUD** — score and lives at the top of the screen
- **Boundaries** — walls built from indestructible bricks

Architecture: who is responsible for what

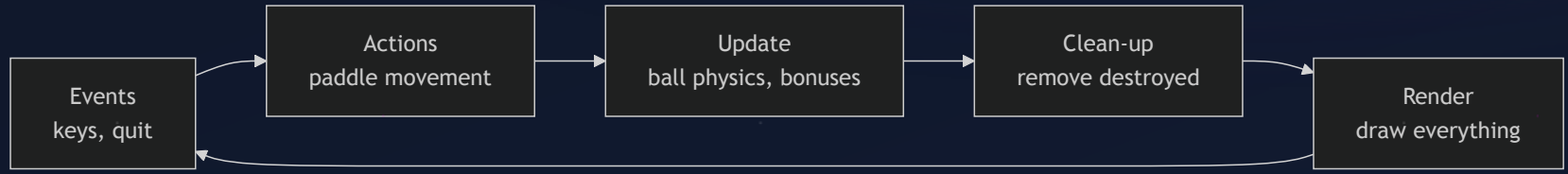
```
arkanoid_pygame/
├── main.py            ← entry point, screen switching
├── settings.py        ← ALL constants in one place
├── game/
│   ├── entities.py    ← Paddle, Ball, Brick, Bonus, LaserBullet
│   ├── level.py       ← level parser for .txt files
│   ├── particles.py   ← particle burst when a brick explodes
│   └── audio.py       ← sounds and music
├── screens/
│   ├── menu.py        ← main menu
│   ├── game_screen.py ← all the gameplay logic
│   └── ...            ← settings, win, game over
└── levels/level1.txt  ← a level as plain text
```

💡 The rule: **logic, settings and screens live separately** —

any game value can be tweaked in a single file, `settings.py`.

The game loop: the heart of every game

A game is a loop that runs **60 times per second**:




```
while True:                        # the main loop
    for event in pygame.event.get(): ...   # events
    paddle.move(keys)                      # actions
    score += _update_balls(...)            # update
    _draw_frame(screen, ...)               # render
    clock.tick(cfg.FPS)                    # exactly 60 FPS
```

The hardest part: bounce physics

Which side of the brick did the ball hit? Find the **smallest overlap**:

```
overlap_left = ball.rect.right - rect.left      # intrusion from the left
overlap_right = rect.right - ball.rect.left     # intrusion from the right
overlap_top = ball.rect.bottom - rect.top       # intrusion from the top
overlap_bottom = rect.bottom - ball.rect.top    # intrusion from the bottom

min_overlap = min(overlap_left, overlap_right, overlap_top, overlap_bottom)

if min_overlap == overlap_top and ball.vy > 0:
    ball.rect.bottom = rect.top   # snap to the edge
    ball.vy *= -1                 # reflect the velocity
elif min_overlap == overlap_left and ball.vx > 0:
    ball.rect.right = rect.left
    ball.vx *= -1
# ... same for bottom and right
```

⚠ Always check the **direction** of the velocity — otherwise the ball gets stuck inside the brick.

Levels are plain text files

levels/level1.txt:

```
0 . 0
1 1 1 1 1 1 1 1 1 1 1 1 1
. 2 2 2 . . 2 2 2 . 2 2
1 1 . 1 1 . 1 1 . 1 1 .
```

- `1`, `2` — brick durability
- `0` — indestructible
- `.` — empty space

The parser — just a few lines:

```
for r, line in enumerate(lines):
    for c, token in enumerate(line.split()):
        if token == '.':
            continue
        if token.isdigit():
            bricks.append(Brick(c, r, int(token)))
```

💡 A new level = a new .txt file.

Anyone can design levels, even without knowing Python!

Power-ups: all 7 of them

	Letter	Type	Effect
	E	extend	paddle ×2 wider
	M	multiball	+1 ball
	L	laser	shoot with spacebar
	1	extra_life	+1 life
	S	shrink	narrower paddle — **mine**
	+	speed_up	faster ball — **mine**
	–	speed_down	slower ball — **mine**

My homework: 3 new power-ups

Slowing the ball down:

```
def speed_down(self) -> None:
    if abs(self.vx) > cfg.MIN_BALL_SPEED:
        self.vx -= 1 if self.vx > 0 else -1
    if abs(self.vy) > cfg.MIN_BALL_SPEED:
        self.vy -= 1 if self.vy > 0 else -1
```

What this small feature taught me:

- ⚠ the ball must **never stop** → a minimum speed limit
- ⚠ the sign of velocity = direction → change only the magnitude
- ⚠ the paddle must not "teleport" when shrinking → keep its center

🎯 The main lesson: even a "simple" feature drags edge cases along with it.

Demo: play right inside the slide!

Score: 0 · Click to start

▶ INSERT COIN - CLICK TO PLAY

The same gameplay, rewritten in JavaScript + canvas for this presentation

What I learned

- **The game loop** — how 60 frames per second become a game
- **Collisions** — AABB overlaps, bounces, edge cases
- **Architecture** — separating settings, logic and rendering
- **Git** — branches, commits, working with someone else's repo
- **Deeper Python** — type hints, classes, modules and packages
- **Working with AI** — how to give it tasks and verify the results

Tools

Code

- Python 3.11
- pygame 2.6.1
- VS Code

Process

- Git + GitHub
- venv
- ruff / pylint

AI

- Claude Code — helped with code and this presentation
- Slidev — slides from markdown

Honestly: AI helped me understand and verify things, but I ran and tested every power-up myself.

Thank you!

Questions?

Game code on GitHub

Slides: Slidev · the mini-game on slide 11 is playable